RUNGTA INTERNATIONAL SKILLS UNIVERSITY

School of Information Technology
Bachelor of Computer Application

PROJECT REPORT

Problem Statement No. 72

GROCERY STORE MANAGEMENT SYSTEM

Language: Python 3

Submitted by	Monali Kumari
Enrollment No.	00582_____
Semester / Section	1st/A_____
Submitted to	_____
Date of Submission	_____

1. Introduction

A grocery store needs a fast and accurate way to prepare customer bills and keep track of its stock. Calculating bills by hand is slow and error-prone, especially when discounts and taxes are involved. This project is a console-based billing application written in Python 3 that automates the whole process.

The cashier selects items and quantities from the inventory. The program calculates the subtotal, discount, GST and the final bill, reduces the stock, and saves every bill in a file.

2. Problem Statement

Build a console billing application for a grocery store. The cashier selects items and quantities from the inventory; the program calculates the subtotal, discount, GST and final bill, reduces the stock and saves every bill in a file.

The starting inventory is given below. A 10% discount is given if the subtotal is Rs. 1000 or more. GST is 5% on the amount after discount. Bills are saved in bills.txt.

Item	Price	Stock
Rice	Rs. 50 per kg	100
Sugar	Rs. 45 per kg	50
Milk	Rs. 30 per L	40
Oil	Rs. 150 per L	20

3. Objectives

- Store the inventory as a dictionary and the current cart as a list of items.
- Calculate the subtotal, discount, GST and total using the given rules.
- Check that the item exists and that enough stock is available before adding it to the cart.
- Reduce the stock after billing and save the updated stock.
- Append every final bill to bills.txt.

4. Tools and Concepts Used

Item	Details
Language	Python 3
Concepts	Lists, Dictionaries, Functions, Billing, File Handling
Modules	json (saving the inventory), os (checking if a file exists), datetime (date and time on bills)
Files created	inventory.json (saved stock), bills.txt (all bills)

Item	Details
Editor / Platform	Visual Studio Code, run from the console

5. Billing Rules

Step	Rule
Subtotal	Sum of (price x quantity) for every item in the cart
Discount	10% of the subtotal if the subtotal is Rs. 1000 or more, otherwise 0
GST	5% of (subtotal - discount)
Total	(subtotal - discount) + GST

Example: Oil x 4 (Rs. 600) and Rice x 8 (Rs. 400) give a subtotal of Rs. 1000.00. Discount = 10% of 1000 = 100.00. GST = 5% of 900 = 45.00. Total = 900 + 45 = Rs. 945.00.

6. Program Design

6.1 Data Structures

- Inventory: a dictionary in the form {item_name: {"price": ..., "stock": ...}}.
- Cart: a list of (item_name, quantity) pairs for the current bill.

6.2 Algorithm

- Start the program and load the inventory (saved stock if available, otherwise the default).
- Show the menu: 1. New bill, 2. View stock, 3. Exit.
- For a new bill, repeatedly ask for an item name and quantity until the cashier types 'done'.
- Reject unknown items, invalid or non-positive quantities, and quantities above the available stock.
- Calculate the subtotal, discount, GST and total and print the bill.
- Reduce the stock, save the inventory to inventory.json and append the bill to bills.txt.
- Return to the menu until the user chooses Exit.

6.3 Functions

Function	Description
load_inventory()	Loads saved stock from inventory.json, or returns the default inventory if the file does not exist.
save_inventory()	Writes the updated stock to inventory.json.

Function	Description
find_item()	Case-insensitive search for an item; returns the real item name or None.
view_stock()	Prints every item with its stock and price.
calculate_bill()	Returns the subtotal, discount, GST and total for the cart.
discount_label()	Returns 'Discount (10%)' when a discount is applied, otherwise 'Discount'.
save_bill()	Appends the final bill with date and time to bills.txt.
new_bill()	Takes items, validates input, prints the bill and updates the stock.
main()	Shows the main menu and runs the program loop.

7. Validations

- Item names are matched without considering upper or lower case (rice, RICE and Rice are the same item).
- If the item is not in the inventory, a message is shown and the cashier is asked again.
- A quantity that is not a whole number, or is zero or negative, is rejected.
- If the same item is added more than once, the stock check includes the quantity already in the cart.
- If the cart is empty when the cashier types 'done', the bill is cancelled and nothing is saved.

8. Test Cases

Test Case 1: Normal Input

Input	Expected Output
Menu option 1 (New bill) Rice: 2, Milk: 3	Subtotal: 190.00 Discount: 0.00 GST (5%): 9.50 Total: Rs. 199.50

Test Case 2: Boundary / Minimum Input

Input	Expected Output
Menu option 1 (New bill) Oil: 4, Rice: 8 (subtotal exactly 1000)	Subtotal: 1000.00 Discount (10%): 100.00 GST (5%): 45.00 Total: Rs. 945.00

Test Case 3: Invalid Input

Input	Expected Output
Menu option 1 (New bill) Item: Butter, Quantity: 2 Item: Milk, Quantity: 50	Item 'Butter' not found in inventory. Insufficient stock for Milk. Available: 40

Test Case 4: Different Valid Scenario

Input	Expected Output
(a) Menu option 1 (New bill) Sugar: 10, Milk: 5	Subtotal: 600.00 Discount: 0.00 GST (5%): 30.00 Total: Rs. 630.00
(b) Menu option 2 (View stock)	Rice: 100 Sugar: 40 Milk: 35 Oil: 20

Note: the test cases assume a fresh inventory. inventory.json should be deleted before running each test case to reset the stock.

9. Sample Bill Saved in bills.txt

```
==================================
Date: 30-10-2026 01:10:00
Oil x 4
Rice x 8
Subtotal: 1000.00
Discount (10%): 100.00
GST (5%): 45.00
Total: Rs. 945.00
```

10. Conclusion

The Grocery Store Management System meets all the requirements of the problem statement. It calculates the subtotal, discount, GST and final bill correctly, validates the cashier's input, reduces the stock after every sale, and keeps a permanent record of every bill in a file. The program was tested with the four given test cases and produced the expected results.

Possible future improvements include adding new items, restocking from the menu, a graphical interface, and daily sales reports.

Appendix: Source Code (grocery_store.py)

```
import json
import os
from datetime import datetime


# File names used for saving data
INVENTORY_FILE = "inventory.json"   # stores current stock between runs
BILL_FILE = "bills.txt"             # every final bill is appended here


# Billing rules
DISCOUNT_LIMIT = 1000     # discount applies if subtotal >= Rs. 1000
DISCOUNT_PERCENT = 10     # discount percentage
GST_PERCENT = 5           # GST percentage on the discounted amount


# Starting inventory (used only when inventory.json does not exist yet)
DEFAULT_INVENTORY = {
    "Rice":  {"price": 50,  "stock": 100},   # per kg
    "Sugar": {"price": 45,  "stock": 50},    # per kg
    "Milk":  {"price": 30,  "stock": 40},    # per L
    "Oil":   {"price": 150, "stock": 20},    # per L
}



def load_inventory():
    """Load the saved stock from inventory.json if it exists,
    otherwise return a fresh copy of the default inventory."""
    if os.path.exists(INVENTORY_FILE):
        with open(INVENTORY_FILE, "r") as f:
            return json.load(f)
    return json.loads(json.dumps(DEFAULT_INVENTORY))  # a fresh copy



def save_inventory(inventory):
    """Write the current inventory (with updated stock) to inventory.json."""
    with open(INVENTORY_FILE, "w") as f:
        json.dump(inventory, f, indent=4)



def find_item(inventory, name):
    """Case-insensitive search for an item.
    Returns the real item name (e.g. 'Rice') or None if not found."""
    for key in inventory:
        if key.lower() == name.strip().lower():
            return key
    return None



def view_stock(inventory):
    """Print every item with its remaining stock and price."""
    print("\n--- Current Stock ---")
    for name, info in inventory.items():
        print(f"{name}: {info['stock']}  (Price: Rs. {info['price']})")



def calculate_bill(cart, inventory):
    """Calculate the bill for the items in the cart.
    - subtotal: sum of price x quantity
    - discount: 10% of subtotal if subtotal >= 1000, else 0
    - gst: 5% of the amount after discount
    - total: amount after discount + gst
    Returns (subtotal, discount, gst, total)."""
    subtotal = sum(inventory[item]["price"] * qty for item, qty in cart)
    discount = subtotal * DISCOUNT_PERCENT / 100 if subtotal >= DISCOUNT_LIMIT else 0.0
    after_discount = subtotal - discount
    gst = after_discount * GST_PERCENT / 100
```

```
    total = after_discount + gst
    return subtotal, discount, gst, total


def discount_label(discount):
    """Return the label for the discount line.
    Shows the percentage only when a discount was actually given,
    e.g. 'Discount (10%)', otherwise just 'Discount'."""
    if discount > 0:
        return f"Discount ({DISCOUNT_PERCENT}%)"
    return "Discount"


def save_bill(cart, subtotal, discount, gst, total):
    """Append the final bill (date, items, subtotal, discount, GST, total)
    to bills.txt. Old bills are kept because the file is opened in append mode."""
    with open(BILL_FILE, "a") as f:
        f.write("=" * 35 + "\n")
        f.write(f"Date: {datetime.now().strftime('%d-%m-%Y %H:%M:%S')}\n")
        for item, qty in cart:
            f.write(f"{item} x {qty}\n")
        f.write(f"Subtotal: {subtotal:.2f}\n")
        f.write(f"{discount_label(discount)}: {discount:.2f}\n")
        f.write(f"GST ({GST_PERCENT}%): {gst:.2f}\n")
        f.write(f"Total: Rs. {total:.2f}\n")


def new_bill(inventory):
    """Create a new bill:
    1. Keep asking for item + quantity until the cashier types 'done'.
    2. Validate the item name, the quantity and the available stock.
    3. Calculate and print the bill.
    4. Reduce the stock, then save the inventory and the bill to files."""
    cart = []  # list of (item_name, quantity)
    print("\n--- New Bill --- (type 'done' as the item name to finish)")

    while True:
        name = input("Item: ").strip()
        if name.lower() == "done":
            break

        # Check that the item exists in the inventory
        item = find_item(inventory, name)
        if item is None:
            print(f"Item '{name}' not found in inventory.")
            continue

        # Check that the quantity is a positive whole number
        try:
            qty = int(input("Quantity: "))
        except ValueError:
            print("Invalid quantity. Enter a whole number.")
            continue
        if qty <= 0:
            print("Quantity must be greater than 0.")
            continue

        # Check stock, counting what is already in the cart for this item
        already_in_cart = sum(q for i, q in cart if i == item)
        available = inventory[item]["stock"] - already_in_cart
        if qty > available:
            print(f"Insufficient stock for {item}. Available: {available}")
            continue

        cart.append((item, qty))
        print(f"Added {qty} x {item}")
```

```
    if not cart:
        print("No items in cart. Bill cancelled.")
        return

    subtotal, discount, gst, total = calculate_bill(cart, inventory)

    # Print the bill on screen
    print("\n------ BILL ------")
    for item, qty in cart:
        print(f"{item} x {qty} = {inventory[item]['price'] * qty:.2f}")
    print(f"Subtotal: {subtotal:.2f}")
    print(f"{discount_label(discount)}: {discount:.2f}")
    print(f"GST ({GST_PERCENT}%): {gst:.2f}")
    print(f"Total: Rs. {total:.2f}")

    # Reduce stock, then save everything
    for item, qty in cart:
        inventory[item]["stock"] -= qty
    save_inventory(inventory)
    save_bill(cart, subtotal, discount, gst, total)
    print("Bill saved to bills.txt")


def main():
    """Show the main menu in a loop until the user chooses Exit."""
    inventory = load_inventory()
    while True:
        print("\n===== GROCERY STORE =====")
        print("1. New bill")
        print("2. View stock")
        print("3. Exit")
        choice = input("Choose an option: ").strip()

        if choice == "1":
            new_bill(inventory)
        elif choice == "2":
            view_stock(inventory)
        elif choice == "3":
            print("Goodbye!")
            break
        else:
            print("Invalid option. Try again.")


main()
```